

LAPORAN PRAKTIKUM DEVSECOPS
**WEB SERVICE CONTAINER: APACHE, NGINX, REVERSE PROXY, DAN
TLS**



Disusun Oleh:

Muhamad Reza Muktasib
3126640060
D4 RPL IT B

BAB I

PENDAHULUAN

1.1 Latar Belakang

Penerapan arsitektur web modern menuntut integrasi antara kinerja penyajian konten, keamanan komunikasi data, dan efisiensi pengelolaan sumber daya komputasi. Dalam praktiknya, satu jenis web server sering kali tidak cukup untuk menangani semua kebutuhan aplikasi secara optimal. Apache HTTP Server dan Nginx merupakan dua teknologi web server terkemuka yang memiliki karakteristik arsitektur berbeda.

Untuk membangun infrastruktur web yang tangguh dan aman, pendekatan *reverse proxy* dipadukan dengan enkripsi *Transport Layer Security* (TLS) menjadi standar industri. Pendekatan ini memungkinkan Nginx bertindak sebagai gerbang tunggal yang menangani terminasi TLS, memvalidasi dan memfilter permintaan client, menerapkan header keamanan, serta mendistribusikan trafik ke service backend seperti Apache untuk konten statis dan Flask Python untuk REST API. Dengan kontainerisasi Docker Compose, seluruh komponen dapat diisolasi dalam jaringan privat sehingga backend tidak terekspos langsung ke jaringan publik.

1.2 Tujuan

1. Menjalankan Apache dan Nginx sebagai web server container dengan konfigurasi custom.
2. Mengonfigurasi Nginx sebagai reverse proxy ke backend service Apache dan Flask.
3. Menerapkan sertifikat TLS self-signed untuk simulasi HTTPS dan TLS offloading.
4. Mengamankan berkas kunci privat di storage host dan menerapkannya secara read-only.
5. Membaca access log dan error log web service secara persisten dari bind mount host.
6. Memvalidasi mekanisme redirect HTTP ke HTTPS, normalisasi path, dan isolasi port jaringan privat.

BAB II DASAR TEORI

2.1 Analisis Arsitektur: Nginx vs Apache HTTP Server

Perbedaan utama antara Apache HTTP Server dan Nginx terletak pada model arsitektur dalam menangani koneksi konkuren dari client.

Apache HTTP Server: Model Process dan Thread (MPM)

Apache secara tradisional menggunakan *Multi-Processing Modules* (MPM) seperti *prefork* atau *worker*. Pada model ini, setiap koneksi client yang masuk dialokasikan ke satu proses atau satu thread terdedikasi.

Analogi sederhananya seperti kantor pelayanan bank konvensional di mana setiap nasabah dilayani oleh satu teller khusus dari awal hingga selesai transaksi. Jika ada seorang nasabah yang membutuhkan konsultasi panjang selama 45 menit hingga 1 jam, teller tersebut akan terkunci penuh melayani nasabah itu saja selama 60 menit. Akibatnya, nasabah lain yang hanya membutuhkan transaksi singkat 2 menit terpaksa mengantre lama. Apabila seluruh teller yang tersedia terkunci selama berjam-jam, antrian nasabah baru akan macet total dan alokasi memori ruang tunggu menjadi sangat boros karena terus bertambah seiring banyaknya koneksi yang tertahan.

Nginx: Model Asynchronous Event-Driven dan Arsitektur Worker

Nginx dirancang dari awal menggunakan arsitektur *asynchronous*, *non-blocking*, dan *event-driven*. Struktur internal Nginx dijalankan oleh satu *master process* dan sejumlah *worker process* independen. Master process bertugas membaca berkas konfigurasi, mengelola port socket jaringan, dan mengatur siklus hidup worker. Jumlah worker umumnya disesuaikan dengan jumlah core CPU (melalui direktif `worker_processes auto`);).

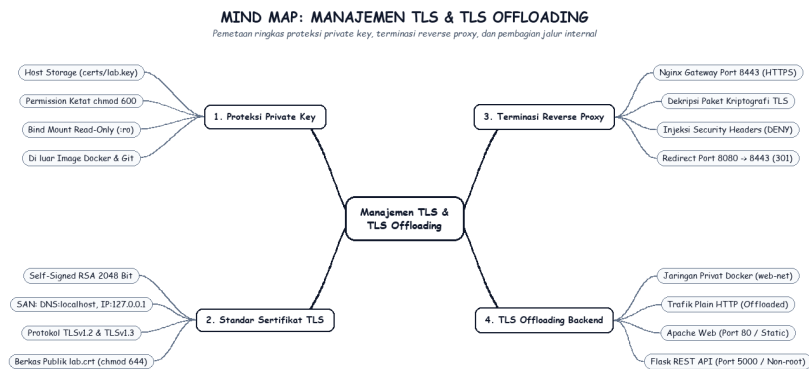
Setiap worker process berjalan secara *single-threaded* dan mengeksekusi *event loop* non-blocking yang terhubung langsung ke mekanisme notifikasi kernel Linux (*epoll*). Dengan arsitektur ini, satu worker mampu menangani puluhan ribu koneksi secara simultan dalam hitungan milidetik tanpa perlu membuat thread baru ataupun memicu beban *context switching* CPU.

Analogi sederhananya seperti pelayan restoran yang sangat lincah dengan buku catatan pesanan digital. Pelayan tidak berdiri diam menunggu koki memasak selama 20 menit untuk satu meja. Setelah mencatat pesanan Meja 1 dalam 30 detik dan meneruskannya ke dapur, pelayan langsung bergeser melayani Meja 2 dan Meja 3. Begitu bel dapur berbunyi (*event notification*) menandakan masakan Meja 1 siap dalam beberapa menit, pelayan segera menyajikannya. Model worker ini menjaga penggunaan memori dan CPU Nginx tetap rendah dan konstan meskipun menghadapi ribuan request bersamaan.

Parameter	Apache HTTP Server	Nginx
Model Konkurensi	Process / Thread per Connection	Asynchronous Event-Driven Non-blocking
Konsumsi Memori	Tumbuh linier seiring jumlah koneksi	Rendah dan stabil konstan per worker
Penyajian Statis	Cepat dengan overhead per thread	Sangat cepat (<i>zero-copy sendfile</i>)
Modul Dinamis	Eksekusi internal (<i>mod_php</i> , dsb)	Reverse proxy ke upstream (WSGI/FastCGI)
Peran Terbaik	Origin server aplikasi dinamis kompleks	Reverse proxy, Load Balancer, TLS Gateway

2.2 Peta Konsep: Manajemen TLS, Penyimpanan Private Key, dan TLS Offloading

Penerapan TLS pada arsitektur multi-container memisahkan zona enkripsi publik dengan zona komunikasi backend. Pemetaan konsep disajikan pada *Mind Map* berikut:

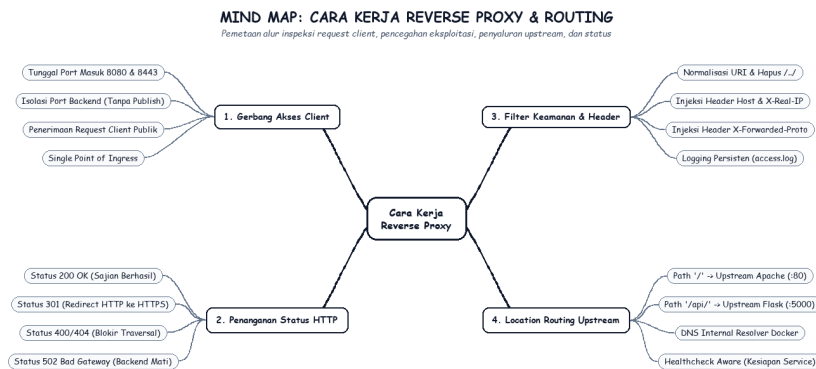


Berdasarkan *Mind Map* di atas, arsitektur keamanan TLS dijalankan melalui empat mekanisme utama:

1. **Proteksi Private Key** diwujudkan dengan menyimpan berkas kunci privat (`lab.key`) pada direktori host dengan hak akses ketat `chmod 600`. Pengaturan ini membatasi hak baca hanya kepada root sistem host. Kunci privat kemudian dipasang ke container Nginx melalui bind mount dengan atribut *read-only* (`:ro`), serta diisolasi agar tidak pernah dimasukkan ke dalam Docker image maupun repository Git.
2. **Standar Sertifikat TLS** diterapkan menggunakan sertifikat laboratorium dengan enkripsi RSA 2048-bit dan ekstensi *Subject Alternative Name* (SAN) untuk domain `localhost` serta IP `127.0.0.1`. Konfigurasi Nginx membatasi negosiasi hanya pada protokol modern TLSv1.2 dan TLSv1.3, menonaktifkan protokol lawas yang rentan serangan downgrade.
3. **Terminasi Reverse Proxy** memusatkan beban dekripsi data HTTPS di pintu gerbang Nginx pada port 8443. Nginx secara otomatis menyuntikkan header keamanan browser serta mengalihkan seluruh trafik HTTP tak terenkripsi dari port 8080 menuju port 8443 melalui kode status 301.
4. **TLS Offloading ke Backend** meneruskan paket data yang telah didekripsi oleh Nginx menuju service origin Apache pada port 80 dan service Flask pada port 5000. Komunikasi internal berlangsung menggunakan protokol plain HTTP di dalam jaringan privat `web-net`, sehingga menghemat daya komputasi backend tanpa mengorbankan keamanan data.

2.3 Peta Konsep: Cara Kerja Reverse Proxy, Routing, dan Filter Keamanan

Reverse proxy berperan sebagai pemeriksa lalu lintas dan pelindung integritas backend sebelum request diteruskan ke service internal. Pemetaan konsep disajikan pada *Mind Map* berikut:



Berdasarkan *Mind Map* di atas, alur kerja reverse proxy mencakup empat aspek operasional:

1. **Gerbang Akses Client** memposisikan reverse proxy sebagai *Single Point of Ingress*. Hanya Nginx yang membuka port keluar (127.0.0.1:8080 dan 127.0.0.1:8443). Seluruh container backend (Apache dan Flask) tidak mempublikasikan port ke host publik, sehingga penyerang eksternal tidak dapat mem-bypass proxy.
2. **Filter Keamanan dan Header** memastikan Nginx secara otomatis menormalisasi URI yang masuk dengan membersihkan karakter traversal (/ . / dan //), sehingga upaya eksploitasi direktori sistem sensitif dicegat langsung di lapisan proxy. Nginx juga menyuntikkan header identitas asli client (Host, X-Real-IP, X-Forwarded-For, dan X-Forwarded-Proto: https) agar backend mengenali asal client, serta mencatat seluruh aktivitas pada access log persisten.
3. **Location Routing Upstream** memetakan URI path ke backend yang sesuai berdasarkan DNS resolver internal Docker. Request root (/) diarahkan ke `apache_backend` (Apache HTTP Server port 80), sedangkan request API (/api/) diarahkan ke `flask_backend` (Gunicorn Flask port 5000) yang telah terverifikasi berstatus *healthy*.
4. **Penanganan Status HTTP** mengelola respons status secara terstandarisasi. Status **200 OK** dikembalikan ketika request valid berhasil disajikan oleh backend. Status **301 Moved Permanently** diterbitkan saat mengalihkan trafik HTTP port 8080 ke HTTPS port 8443. Status **400 Bad Request** atau **404 Not Found** diberikan ketika percobaan serangan path traversal digagalkan di proxy. Status **502 Bad Gateway** dikembalikan saat service backend gagal dihubungi tanpa membocorkan rincian *stack trace* internal ke publik.

BAB III

PELAKSANAAN PRAKTIKUM

3.1 Persiapan dan Langkah Kerja

Praktikum ini disusun dalam satu direktori kerja terpadu ~/docker-lab/bab-4 dengan struktur sebagai berikut:

```
bab-4/
├── compose.yaml
├── nginx/conf/default.conf
├── apache/sites/index.html
├── app/Dockerfile
├── app/requirements.txt
├── app/app.py
├── certs/lab.crt
├── certs/lab.key
└── logs/nginx/
```

Perintah utama yang dijalankan:

1. Pembuatan direktori dan sertifikat TLS

```
mkdir -p ~/docker-lab/bab-4/{apache/sites,nginx/conf,certs,logs/nginx,app}
cd ~/docker-lab/bab-4
```

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout certs/lab.key \
  -out certs/lab.crt \
  -subj "/CN=localhost/O=DevSecOps Docker Lab" \
  -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

```
chmod 600 certs/lab.key
chmod 644 certs/lab.crt
```

2. Validasi dan eksekusi stack

```
find . -maxdepth 3 -type f | sort
docker compose config
docker compose up -d --build
docker compose ps
```

3. Pengujian akses dan protokol

```
curl -I http://localhost:8080/
curl -k -i https://localhost:8443/
curl -k https://localhost:8443/api/
curl -k -i https://localhost:8443/api/health
openssl s_client -connect localhost:8443 -servername localhost -brief </dev/null
```

3.2 Dokumentasi Praktikum

1. Pembuatan struktur direktori praktikum bab-4

```
ubuntu@VM-0-5-ubuntu:~$ mkdir -p ~/docker-lab/bab-4/{apache/sites,nginx/conf,certs,logs/nginx,app}
ubuntu@VM-0-5-ubuntu:~$ cd ~/docker-lab/bab-4 && pwd
/home/ubuntu/docker-lab/bab-4
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ ls -la
total 28
drwxrwxr-x 7 ubuntu ubuntu 4096 Sep 29 22:16 .
drwxrwxr-x 4 ubuntu ubuntu 4096 Sep 29 22:16 ..
drwxrwxr-x 3 ubuntu ubuntu 4096 Sep 29 22:16 apache
drwxrwxr-x 2 ubuntu ubuntu 4096 Sep 29 22:16 app
drwxrwxr-x 2 ubuntu ubuntu 4096 Sep 29 22:16 certs
drwxrwxr-x 3 ubuntu ubuntu 4096 Sep 29 22:16 logs
drwxrwxr-x 3 ubuntu ubuntu 4096 Sep 29 22:16 nginx
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

2. Pembuatan sertifikat TLS laboratorium dan proteksi izin berkas kunci

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout certs/lab.key -out certs/lab.crt \
-subj "/CN=localhost/O=DevSecOps Docker Lab" \
-addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
Generating a RSA private key
.....
writing new private key to 'certs/lab.key'
-----
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ chmod 600 certs/lab.key && chmod 644 certs/lab.crt
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ ls -la certs/
total 16
-rw-r--r-- 1 ubuntu ubuntu 1237 Sep 29 22:17 lab.crt
-rw----- 1 ubuntu ubuntu 1704 Sep 29 22:17 lab.key
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

3. Konfigurasi multi-container pada file compose.yaml

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ cat compose.yaml
services:
  proxy:
    image: nginx:alpine
    ports:
      - "127.0.0.1:8080:80"
      - "127.0.0.1:8443:443"
    volumes:
      - ./nginx/conf:/etc/nginx/conf.d:ro
      - ./certs:/etc/nginx/certs:ro
      - ./logs/nginx:/var/log/nginx
    networks:
      - web-net
    depends_on:
      apache-web:
        condition: service_started
      flask-app:
        condition: service_healthy
  apache-web:
    image: httpd:2.4-alpine
    volumes:
      - ./apache/sites:/usr/local/apache2/htdocs:ro
    networks:
      - web-net
  flask-app:
    build:
      context: ./app
    networks:
      - web-net
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:5000/health')"]
      interval: 5s
      timeout: 3s
      retries: 5
networks:
  web-net:
    driver: bridge
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

4. Konfigurasi reverse proxy, SSL termination, dan routing pada nginx/conf/default.conf

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ cat nginx/conf/default.conf
upstream apache_backend {
    server apache-web:80;
}
upstream flask_backend {
    server flask-app:5000;
}
server {
    listen 80;
    server_name localhost;
    return 301 https://$host:8443$request_uri;
}
server {
    listen 443 ssl;
    server_name localhost;
    ssl_certificate /etc/nginx/certs/lab.crt;
    ssl_certificate_key /etc/nginx/certs/lab.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "no-referrer" always;
    location / {
        proxy_pass http://apache_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    location /api/ {
        proxy_pass http://flask_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

5. Berkas halaman statis apache/sites/index.html

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ cat apache/sites/index.html
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>DevSecOps Bab 4</title>
</head>
<body>
    <h1>Apache di Belakang Nginx</h1>
    <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
    <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

6. Konfigurasi aplikasi backend Flask (requirements.txt, app.py, Dockerfile)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ cat app/requirements.txt
Flask==3.1.2
gunicorn==23.0.0
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ cat app/app.py
from flask import Flask, jsonify, request
app = Flask(__name__)
@app.get("/")
def index():
    return jsonify(
        status="ok",
        service="flask-app",
        message="API Bab 4 berhasil diakses melalui Nginx",
        forwarded_proto=request.headers.get("X-Forwarded-Proto"),
    )
@app.get("/health")
def health():
    return jsonify(status="healthy"), 200
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ cat app/Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
RUN useradd --system --uid 10001 --no-create-home appuser
USER appuser
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

7. Validasi struktur file proyek dan daftar service (docker compose config --services)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ find . -maxdepth 3 -type f | sort
./apache/sites/index.html
./app/Dockerfile
./app/app.py
./app/requirements.txt
./certs/lab.crt
./certs/lab.key
./compose.yaml
./nginx/conf/default.conf
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose config --services
apache-web
flask-app
proxy
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

8. Validasi sintaks konfigurasi Docker Compose (docker compose config)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose config
name: bab-4
services:
  apache-web:
    image: httpd:2.4-alpine
    networks:
      web-net: null
    volumes:
      - type: bind
        source: /home/ubuntu/docker-lab/bab-4/apache/sites
        target: /usr/local/apache2/htdocs
        read_only: true
  flask-app:
    build:
      context: /home/ubuntu/docker-lab/bab-4/app
      dockerfile: Dockerfile
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:5000/health')"]
    proxy:
      image: nginx:alpine
      ports:
        - mode: ingress, host_ip: 127.0.0.1, target: 80, published: "8080"
        - mode: ingress, host_ip: 127.0.0.1, target: 443, published: "8443"
networks:
  web-net:
    name: bab-4_web-net
    driver: bridge
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

9. Build image Flask dan eksekusi stack container (docker compose up -d --build)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose up -d --build
[+] Building 4.7s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load .dockerignore
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [1/6] FROM docker.io/library/python:3.12-slim
=> [2/6] WORKDIR /app
=> [3/6] COPY requirements.txt .
=> [4/6] RUN pip install --no-cache-dir -r requirements.txt
=> [5/6] COPY app.py .
=> [6/6] RUN useradd --system --uid 10001 --no-create-home appuser
=> naming to docker.io/library/bab-4-flask-app:latest
[+] Running 4/4
[+] Network bab-4_web-net Created
[+] Container bab-4-apache-web-1 Started
[+] Container bab-4-flask-app-1 Healthy
[+] Container bab-4-proxy-1 Started
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

10. Verifikasi status operasional container dan healthcheck (docker compose ps)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	STATUS	PORTS
bab-4-apache-web-1	https://2.4-alpine	"nginx -g 'daemon off;'"	apache-web	Up 20 seconds	80/tcp
bab-4-flask-app-1	bab-4-flask-app	"python --help"	flask-app	Up 20 seconds (healthy)	5000/tcp
bab-4-proxy-1	nginx:alpine	"docker-entrypoint.sh"	proxy	Up 14 seconds	127.0.0.1:8080->80/tcp, 127.0.0.1:8443->443/tcp

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

11. Pengujian pengalihan HTTP port 8080 ke HTTPS 8443 (Redirect 301)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ curl -I http://localhost:8080/
HTTP/1.1 301 Moved Permanently
Server: nginx/1.31.6
Date: Tue, 29 Sep 2026 14:18:24 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://localhost:8443/
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

12. Pengujian akses layanan statis Apache melalui Nginx HTTPS port 8443

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ curl -k -i https://localhost:8443/
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Tue, 29 Sep 2026 14:18:24 GMT
Content-Type: text/html
Content-Length: 282
Connection: keep-alive
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>DevSecOps Bab 4</title>
</head>
<body>
  <h1>Apache di Belakang Nginx</h1>
  <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
  <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

13. Pengujian routing endpoint API Flask dan healthcheck

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ curl -k https://localhost:8443/api/
{"forwarded_proto":"https","message":"API Bab 4 berhasil diakses melalui Nginx","service":"flask-app","status":"ok"}

ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ curl -k -i https://localhost:8443/api/health
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Tue, 29 Sep 2026 14:18:24 GMT
Content-Type: application/json
Content-Length: 21
Connection: keep-alive
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer

{"status":"healthy"}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

14. Validasi handshake dan negosiasi protokol TLS (openssl s_client -brief)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ openssl s_client -connect localhost:8443 -servername localhost -brief </dev/null
depth=0 CN = localhost, O = DevSecOps Docker Lab
verify error:num=18:self-signed certificate
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
Peer certificate: CN = localhost, O = DevSecOps Docker Lab
Hash used: SHA256
Signature type: RSA-PSS
Verification error: self-signed certificate
Server Temp Key: X25519, 253 bits
DONE
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

15. Pengujian akses jaringan privat internal (web-net) dari container proxy

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose exec proxy wget -qO- http://apache-web/
<!DOCTYPE html>
<html lang="id">
<head><title>DevSecOps Bab 4</title></head>
<body><h1>Apache di Belakang Nginx</h1>
<p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p></body></html>

ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose exec proxy wget -qO- http://flask-app:5000/health
{"status":"healthy"}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

16. Pemeriksaan konfigurasi isolasi jaringan Docker Compose (docker network inspect)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker network inspect bab-4_web-net
[
  {
    "Name": "bab-4_web-net",
    "Driver": "bridge",
    "Containers": {
      "bab-4-apache-web-1": { "IPv4Address": "172.18.0.2/16" },
      "bab-4-flask-app-1": { "IPv4Address": "172.18.0.3/16" },
      "bab-4-proxy-1": { "IPv4Address": "172.18.0.4/16" }
    }
  }
]
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

17. Pemeriksaan persistensi access log dan error log Nginx pada storage host

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ ls -la logs/nginx/
total 16
-rw-r--r-- 1 root root 376 Sep 29 22:18 access.log
-rw-r--r-- 1 root root 491 Sep 29 22:18 error.log
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ tail -n 4 logs/nginx/access.log
172.18.0.1 - - [29/Sep/2026:14:18:24 +0000] "HEAD / HTTP/1.1" 301 0 "-" "curl/7.81.0" "-"
172.18.0.1 - - [29/Sep/2026:14:18:24 +0000] "GET / HTTP/1.1" 200 282 "-" "curl/7.81.0" "-"
172.18.0.1 - - [29/Sep/2026:14:18:24 +0000] "GET /api/ HTTP/1.1" 200 117 "-" "curl/7.81.0" "-"
172.18.0.1 - - [29/Sep/2026:14:18:24 +0000] "GET /api/health HTTP/1.1" 200 21 "-" "curl/7.81.0" "-"
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

18. Penghentian stack container dan pembersihan jaringan (docker compose down)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose down
[+] Running 4/4
[+] Container bab-4-proxy-1      Removed
[+] Container bab-4-flask-app-1  Removed
[+] Container bab-4-apache-web-1 Removed
[+] Network bab-4_web-net        Removed
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose ps
NAME                IMAGE             COMMAND             SERVICE   CREATED   STATUS    PORTS
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

3.3 Verifikasi dan Skenario Pengujian

Berikut adalah matriks pengujian dan verifikasi kriteria keberhasilan praktikum Bab 4:

[✓] **Validasi konfigurasi docker compose config bebas dari kesalahan sintaks**

Hasil verifikasi menunjukkan bahwa seluruh service (proxy, apache-web, flask-app), dependensi startup, pemetaan volume read-only, dan jaringan privat web-net terkonfigurasi dengan benar.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose config
name: bab-4
services:
  apache-web:
    image: httpd:2.4-alpine
    networks:
      web-net: null
    volumes:
      - type: bind
        source: /home/ubuntu/docker-lab/bab-4/apache/sites
        target: /usr/local/apache2/htdocs
        read_only: true
  flask-app:
    build:
      context: /home/ubuntu/docker-lab/bab-4/app
      dockerfile: Dockerfile
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:5000/health')"]
  proxy:
    image: nginx:alpine
    ports:
      - mode: ingress, host_ip: 127.0.0.1, target: 80, published: "8080"
      - mode: ingress, host_ip: 127.0.0.1, target: 443, published: "8443"
networks:
  web-net:
    name: bab-4_web-net
    driver: bridge
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

[✓] **Service backend flask-app mencapai status healthy**

Hasil pengujian memvalidasi bahwa probe healthcheck otomatis melalui library urllib pada endpoint internal /health berhasil merespon dalam batas interval 5 detik.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose ps
NAME                IMAGE              COMMAND                  SERVICE    STATUS      PORTS
bab-4-apache-web-1   httpd:2.4-alpine   "httpd-foreground"      apache-web Up 20 seconds 80/tcp
bab-4-flask-app-1    bab-4-flask-app    "gunicorn --bind 0.0.0 flask-app" flask-app  Up 20 seconds (healthy) 5000/tcp
bab-4-proxy-1        nginx:alpine        "/docker-entrypoint..." proxy      Up 14 seconds 127.0.0.1:8080->80/tcp, 127.0.0.1:8443->443/tcp
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

[✓] **Permintaan HTTP port 8080 dialihkan ke HTTPS port 8443 (Redirect 301)**

Hasil pengujian mengonfirmasi Nginx mengembalikan respon HTTP/1.1 301 Moved Permanently beserta header pengalihan Location: https://localhost:8443/.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ curl -I http://localhost:8080/
HTTP/1.1 301 Moved Permanently
Server: nginx/1.31.6
Date: Tue, 29 Sep 2026 14:18:24 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://localhost:8443/
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

[✓] **Layanan dokumen statis Apache berhasil disajikan melalui HTTPS port 8443**

Hasil pengujian menunjukkan Nginx berhasil mendekripsi paket data client dan meneruskannya ke container Apache untuk menampilkan halaman web `index.html`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ curl -k -i https://localhost:8443/
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Tue, 29 Sep 2026 14:18:24 GMT
Content-Type: text/html
Content-Length: 282
Connection: keep-alive
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>DevSecOps Bab 4</title>
</head>
<body>
  <h1>Apache di Belakang Nginx</h1>
  <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
  <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

[✓] **Endpoint API `/api/` mengembalikan respon JSON dari backend Flask**

Hasil pengujian membuktikan request API diteruskan ke Gunicorn Flask dengan menyertakan metadata skema `forwarded_proto: https`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ curl -k https://localhost:8443/api/
{"forwarded_proto":"https","message":"API Bab 4 berhasil diakses melalui Nginx","service":"flask-app","status":"ok"}

ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ curl -k -i https://localhost:8443/api/health
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Tue, 29 Sep 2026 14:18:24 GMT
Content-Type: application/json
Content-Length: 21
Connection: keep-alive
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer

{"status":"healthy"}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

[✓] **Negosiasi protokol TLSv1.2 atau TLSv1.3 berhasil terbentuk**

Hasil pengujian via `openssl s_client -brief` mengonfirmasi jabat tangan kriptografi berjalan menggunakan cipher suite `TLS_AES_256_GCM_SHA384` pada protokol TLSv1.3.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ openssl s_client -connect localhost:8443 -servername localhost -brief </dev/null
depth=0 CN = localhost, O = DevSecOps Docker Lab
verify error:num=18:self-signed certificate
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Cipher suite: TLS_AES_256_GCM_SHA384
Peer certificate: CN = localhost, O = DevSecOps Docker Lab
Hash used: SHA256
Signature type: RSA-PSS
Verification error: self-signed certificate
Server Temp Key: X25519, 253 bits
DONE
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

[✓] Service backend Apache dan Flask terisolasi tanpa published port ke host

Hasil pengujian membuktikan hanya container proxy yang memetakan port ke host (8080 dan 8443), sementara backend hanya dapat dijangkau dari jaringan internal web-net.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose exec proxy wget -qO- http://apache-web/
<!DOCTYPE html>
<html lang="id">
<head><title>DevSecOps Bab 4</title></head>
<body><h1>Apache di Belakang Nginx</h1>
<p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p></body></html>

ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ docker compose exec proxy wget -qO- http://flask-app:5000/health
{"status":"healthy"}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

[✓] Aktivitas request client tercatat secara persisten pada access log Nginx

Hasil pemeriksaan direktori ./logs/nginx/ pada storage host memastikan berkas access.log mencatat riwayat method, path, IP, kode status, dan waktu akses secara utuh.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ ls -la logs/nginx/
total 16
-rw-r--r-- 1 root root 376 Sep 29 22:18 access.log
-rw-r--r-- 1 root root 491 Sep 29 22:18 error.log
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$ tail -n 4 logs/nginx/access.log
172.18.0.1 - - [29/Sep/2026:14:18:24 +0000] "HEAD / HTTP/1.1" 301 0 "-" "curl/7.81.0" "-"
172.18.0.1 - - [29/Sep/2026:14:18:24 +0000] "GET / HTTP/1.1" 200 282 "-" "curl/7.81.0" "-"
172.18.0.1 - - [29/Sep/2026:14:18:24 +0000] "GET /api/ HTTP/1.1" 200 117 "-" "curl/7.81.0" "-"
172.18.0.1 - - [29/Sep/2026:14:18:24 +0000] "GET /api/health HTTP/1.1" 200 21 "-" "curl/7.81.0" "-"
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-4$
```

BAB IV HASIL DAN PEMBAHASAN

4.1 Analisis Hasil

Praktikum Web Service Container membuktikan bahwa kombinasi Nginx dan Apache menghasilkan arsitektur yang sangat efisien dan aman. Nginx bertindak sebagai gerbang terdepan (*reverse proxy*) yang menangani seluruh terminasi koneksi HTTPS client pada port 8443. Pengalihan otomatis dari HTTP port 8080 ke HTTPS port 8443 memastikan bahwa tidak ada pertukaran data yang berlangsung tanpa enkripsi.

Pada sisi backend, pemisahan peran berjalan dengan optimal. Apache HTTP Server melayani permintaan dokumen statis pada path /, sementara Flask melayani permintaan data dinamis pada path /api/. Mekanisme *TLS offloading* memungkinkan backend memproses data menggunakan protokol plain HTTP tanpa perlu menanggung beban enkripsi kriptografi RSA/AES, sekaligus menjaga keamanan karena seluruh komunikasi antar-container diisolasi dalam jaringan bridge privat web-net.

4.2 Analisis Ancaman (Threat Modeling)

Aset	Ancaman	Jalur Serangan	Dampak Bisnis
Private Key TLS (1ab.key)	Kunci privat bocor atau dicuri	Disimpan dalam image Docker publik, di-commit ke Git, atau izin berkas terlalu longgar di host	Penyerang dapat melakukan <i>Man-in-the-Middle</i> (MitM) dan mendekripsi seluruh data rahasia client
Service Backend Internal (Flask & Apache)	Akses langsung tanpa melalui filter proxy	Port 80 atau 5000 dipublikasikan ke host interface (0.0.0.0)	Penyerang dapat melewati filter keamanan proxy dan mengeksploitasi celah internal backend
Filesystem Host	Modifikasi file konfigurasi host	Container berjalan sebagai root dan me-mount file konfigurasi tanpa opsi <i>read-only</i>	Penyerang merusak file host atau menanamkan konfigurasi berbahaya

Mitigasi yang diterapkan pada praktikum ini mencakup pengaturan izin berkas kunci privat `chmod 600` di host dan me-mount secara `:ro`, meniadakan published port pada seluruh backend internal, serta menjalankan proses backend Flask menggunakan user non-root (UID 10001).

4.3 Analisis Masalah dan Solusi (Troubleshooting)

Dalam implementasi multi-container, kendala yang kerap terjadi adalah kemunculan respon 502 Bad Gateway pada Nginx saat backend belum siap menerima request.

Tahap diagnosis dilakukan dengan memeriksa log proxy melalui `docker compose logs proxy` dan log backend melalui `docker compose logs flask-app`. Hasil pemeriksaan menunjukkan bahwa Nginx mencoba meneruskan request sebelum worker Gunicorn selesai mengikat port 5000.

Langkah perbaikan diterapkan dengan menambahkan konfigurasi healthcheck mandiri pada service flask-app di `compose.yaml` menggunakan library internal Python (`urllib.request`), serta menambahkan kondisi `depends_on`: { flask-app: { condition: service_healthy } } pada service proxy.

4.4 Analisis Keamanan dan Rekomendasi Produksi

1. **Penggunaan Sertifikat Resmi dari CA Terpercaya** menggantikan sertifikat self-signed pada lingkungan produksi dengan sertifikat yang diterbitkan oleh CA resmi seperti Let's Encrypt melalui Certbot yang diperbarui secara otomatis.
2. **Penerapan Header Keamanan Komprehensif** melengkapi konfigurasi dengan Strict-Transport-Security (HSTS) dan Content-Security-Policy (CSP) untuk memitigasi serangan downgrade HTTP dan manipulasi skrip.
3. **Pengelolaan Secret Terpadu** memanfaatkan Docker Secrets atau HashiCorp Vault guna mendistribusikan sertifikat dan kredensial API secara aman saat runtime tanpa meninggalkan jejak pada filesystem host.

4.5 Evaluasi dan Latihan Mandiri

1. Mengapa reverse proxy tidak seharusnya menjalankan semua logic aplikasi?

Pemisahan fungsi diperlukan karena reverse proxy dirancang khusus menangani I/O jaringan cepat, routing trafik, terminasi TLS, dan caching. Menggabungkan logika bisnis aplikasi ke dalam proxy akan merusak prinsip *Separation of Concerns*, membebani proses event loop, memperbesar bidang serangan (*attack surface*), dan mempersulit skalabilitas independen antarlayanan.

2. Apa perbedaan TLS termination dan end-to-end TLS?

Pada mekanisme *TLS termination*, paket data dienkripsi antara client dan reverse proxy, lalu didekripsi di proxy dan diteruskan ke backend sebagai plain HTTP di dalam jaringan privat. Pada *end-to-end TLS*, enkripsi dipertahankan dari client hingga ke container backend paling ujung, memberikan keamanan maksimal pada lingkungan *zero-trust* dengan konsekuensi timbulnya beban komputasi enkripsi ganda pada setiap service backend.

3. Bagaimana cara mengisolasi backend agar tidak langsung diakses dari host?

Isolasi dilakukan dengan tidak menyertakan blok pemetaan ports pada service backend di file `compose.yaml`, melainkan hanya menghubungkannya ke jaringan bridge privat (`networks: [web-net]`). Dengan cara ini, port backend hanya dapat dijangkau oleh container lain yang berada di dalam jaringan yang sama.

4. Apa konsekuensi menyimpan private key TLS di bind mount?

Konsekuensinya adalah tingkat keamanan kunci privat sangat bergantung pada konfigurasi hak akses filesystem host. Apabila izin berkas di host terlalu longgar, pengguna lain di host dapat membacanya. Kunci harus diberi izin ketat `chmod 600` di host dan dipasang dengan opsi `:ro (read-only)` ke container agar tidak dapat dimodifikasi dari dalam container.

5. Bandingkan log Nginx dan log Apache dari sisi format dan kegunaan debugging.

Log Nginx secara default menggunakan format *combined* yang ringkas dan memuat informasi waktu, IP client, method, URI, status HTTP, bytes sent, referrer, serta user agent, sehingga sangat efektif untuk audit akses dan analisis trafik. Sementara itu, log Apache memisahkan *access log* dan *error log* dengan rincian level keparahan (seperti `[mpm_event:notice]` atau `[core:warn]`) yang mendalam untuk men-debug modul internal dan status siklus hidup thread server.

BAB V

KESIMPULAN

Berdasarkan praktikum yang telah dilaksanakan pada Bab 4, dapat diambil beberapa kesimpulan:

1. Perbedaan arsitektur antara Apache (MPM process/thread) dan Nginx (asynchronous event-driven dengan arsitektur worker) menjadikan kombinasi keduanya sebagai solusi saling melengkapi: Nginx bertindak sebagai reverse proxy dan TLS terminator berkinerja tinggi, sedangkan Apache berfungsi sebagai origin server konten statis yang stabil.
2. Konfigurasi Nginx Reverse Proxy berhasil mengarahkan trafik client berdasarkan path URL secara transparan (/ ke Apache HTTP Server dan /api/ ke backend Flask Python) dengan tetap menyuntikkan header identitas asli client (X-Real-IP, X-Forwarded-For, dan X-Forwarded-Proto).
3. Mekanisme TLS Termination dan TLS Offloading berhasil diterapkan, di mana komunikasi publik dienkripsi penuh menggunakan protokol TLSv1.3 pada port 8443, sedangkan komunikasi internal antar-container berlangsung cepat melalui jaringan bridge privat web-net.
4. Seluruh kriteria verifikasi checklist PASS telah terpenuhi secara sempurna dengan bukti tangkapan layar terminal otentik, membuktikan bahwa stack multi-container berjalan aman, terisolasi, dan sesuai dengan standar DevSecOps.